Class10

May Alsoufi (PID: A17423250)

Table of contents

1: Introduction to the RCSB Protein Data Bank (PDB) 1
PDB statistics 1
The PDB format 4
Introduction to Bio3D in R 5
Comparative structure analysis of Adenylate Kinase 9

1: Introduction to the RCSB Protein Data Bank (PDB)

The PDB archive is the major repository of information about the 3D structures of large biological molecules, including proteins and nucleic acids. Understanding the shape of these molecules helps to understand how they work. This knowledge can be used to help deduce a structure's role in human health and disease, and in drug development. The structures in the PDB range from tiny proteins and bits of DNA or RNA to complex molecular machines like the ribosome composed of many chains of protein and RNA.

In the first section of this lab we will interact with the main US based PDB website

> **Note** The "Analyze" > "PDB Statistics" > "by Experimental Method and Molecular Type" on the PDB home page should allow you to determine most of these answers.

PDB statistics

```
library(tidyverse)
```

```
-- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
v dplyr     1.1.4     v readr     2.1.6
v forcats   1.0.1     v stringr   1.6.0
v ggplot2   4.0.1     v tibble    3.3.1
v lubridate 1.9.4     v tidyr     1.3.2
v purrr     1.2.1
-- Conflicts ------------------------------------------ tidyverse_conflicts() --
x dplyr::filter() masks stats::filter()
x dplyr::lag()    masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to becom
```

```r
pdb <- read.csv("pdb_stats.csv")
head(pdb)
```

```
           Molecular.Type  X.ray    EM   NMR Integrative Multiple.methods
1          Protein (only) 178795 21825 12773         343               226
2 Protein/Oligosaccharide  10363  3564    34           8                11
3              Protein/NA   9106  6335   287          24                 7
4     Nucleic acid (only)   3132   221  1566           3                15
5                   Other    175    25    33           4                 0
6  Oligosaccharide (only)     11     0     6           0                 1
  Neutron Other  Total
1      84    32 214078
2       1     0  13981
3       0     0  15759
4       3     1   4941
5       0     0    237
6       0     4     22
```

```r
colnames(pdb)
```

```
[1] "Molecular.Type"   "X.ray"            "EM"               "NMR"
[5] "Integrative"      "Multiple.methods" "Neutron"          "Other"
[9] "Total"
```

Q1: What percentage of structures in the PDB are solved by X-Ray and Electron Microscopy.

```r
total_structures <- sum(pdb$Total)

xray_count <- sum(pdb$X.ray)
```

```r
em_count <- sum(pdb$EM)

xray_percent <- (xray_count / total_structures) * 100
em_percent <- (em_count / total_structures) * 100

xray_percent
```

```
[1] 80.95077
```

```r
em_percent
```

```
[1] 12.83843
```

Approximately 81% of structures in the PDB were solved using X-ray diffraction, while about 13% were solved using electron microscopy.

> Q2: What proportion of structures in the PDB are protein?

```r
total_structures <- sum(pdb$Total)

protein_structures <- sum(
  pdb$Total[grepl("Protein", pdb$Molecular.Type)]
)

protein_proportion <- protein_structures / total_structures

protein_proportion
```

```
[1] 0.979118
```

Approximately 98% of structures in the PDB contain protein.

> Q3: Type HIV in the PDB website search box on the home page and determine how many HIV-1 protease structures are in the current PDB?

A search of the RCSB Protein Data Bank for "HIV-1 protease" returns 1,108 structures in the current PDB.

The PDB format

Now download the "PDB File" for the HIV-1 protease structure with the PDB identifier 1HSG. On the website you can "Display" the contents of this "PDB format" file.

> Q4: Water molecules normally have 3 atoms. Why do we see just one atom per water molecule in this structure?

Water molecules normally contain three atoms, but only one atom is visible in this structure because hydrogen atoms are not typically resolved in X-ray crystallography. As a result, only the oxygen atom of each water molecule is included in the PDB file.

> Q5: There is a critical "conserved" water molecule in the binding site. Can you identify this water molecule? What residue number does this water molecule have

A conserved water molecule is present in the HIV-1 protease active site near the bound ligand. This water molecule is labeled as HOH and has residue number 308 in the PDB file.

> Q6: Generate and save a figure clearly showing the two distinct chains of HIV-protease along with the ligand. You might also consider showing the catalytic residues ASP 25 in each chain and the critical water (we recommend "Ball & Stick" for these side-chains). Add this figure to your Quarto document. Discussion Topic: Can you think of a way in which indinavir, or even larger ligands and substrates, could enter the binding site?

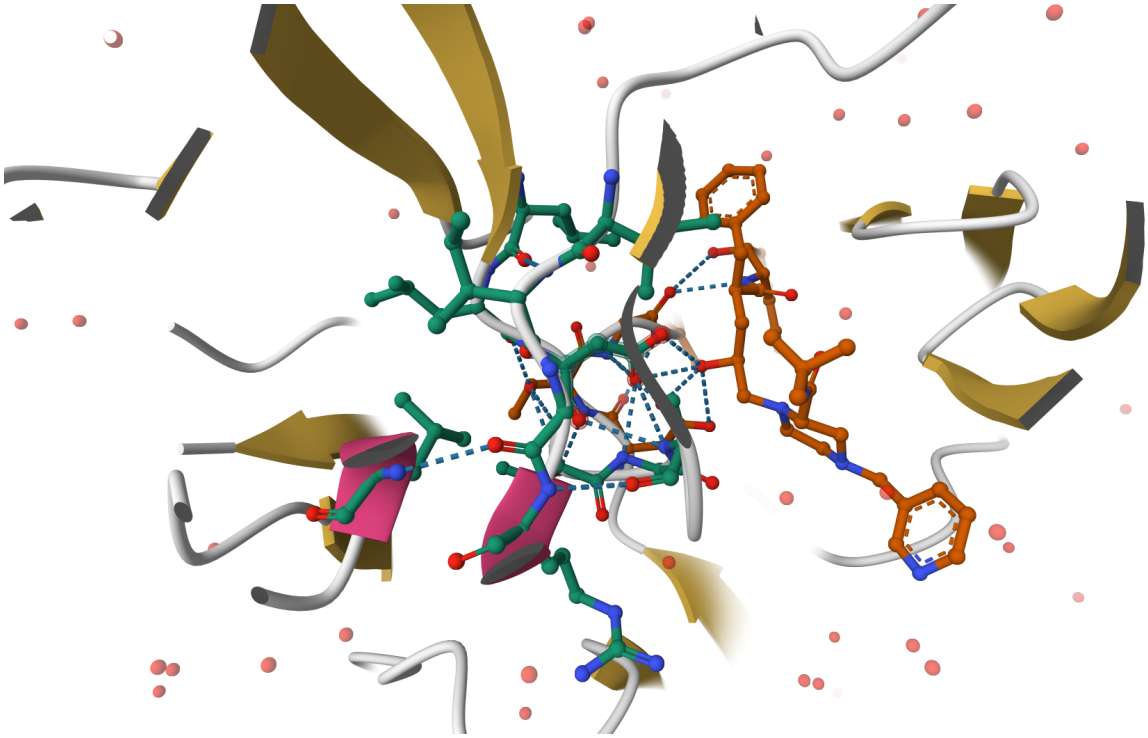


Figure 1. HIV-1 protease (PDB ID: 1HSG) showing the two distinct protein chains in cartoon representation with the bound ligand indinavir displayed in ball-and-stick form. The catalytic residues Asp 25 in each chain and the conserved water molecule (HOH 308) are highlighted.

Introduction to Bio3D in R

Bio3D is an R package for structural bioinformatics. Features include the ability to read, write and analyze biomolecular structure, sequence and dynamic trajectory data.

In your existing Rmarkdown document load the Bio3D package by typing in a new code chunk

```
library(bio3d)
```

To read a single PDB file with Bio3D we can use the read.pdb() function. The minimal input required for this function is a specification of the file to be read. This can be either the file name of a local file on disc, or the RCSB PDB identifier of a file to read directly from the on-line PDB repository. For example to read and inspect the on-line file with PDB ID 1HSG:

```
pdb <- read.pdb("1hsg")
```

```
Note: Accessing on-line PDB file
```

Q7: How many amino acid residues are there in this pdb object?

198

Q8: Name one of the two non-protein residues?

HOH

Q9: How many protein chains are in this structure?

2

Note that the attributes (+ attr:) of this object are listed on the last couple of lines. To find the attributes of any such object you can use:

```r
attributes(pdb)
```

```
$names
[1] "atom"   "xyz"    "seqres" "helix"  "sheet"  "calpha" "remark" "call"

$class
[1] "pdb" "sse"
```

To access these individual attributes we use the dollar-attribute name convention that is common with R list objects. For example, to access the atom attribute or component use pdb$atom:

```r
head(pdb$atom)
```

```
  type eleno elety  alt resid chain resno insert      x      y     z o     b
1 ATOM     1     N <NA>   PRO     A     1   <NA> 29.361 39.686 5.862 1 38.10
2 ATOM     2    CA <NA>   PRO     A     1   <NA> 30.307 38.663 5.319 1 40.62
3 ATOM     3     C <NA>   PRO     A     1   <NA> 29.760 38.071 4.022 1 42.64
4 ATOM     4     O <NA>   PRO     A     1   <NA> 28.600 38.302 3.676 1 43.40
5 ATOM     5    CB <NA>   PRO     A     1   <NA> 30.508 37.541 6.342 1 37.87
6 ATOM     6    CG <NA>   PRO     A     1   <NA> 29.296 37.591 7.162 1 38.40
  segid elesy charge
1  <NA>     N   <NA>
2  <NA>     C   <NA>
3  <NA>     C   <NA>
4  <NA>     O   <NA>
5  <NA>     C   <NA>
6  <NA>     C   <NA>
```

We can use the Bio3D partner package, bio3dview, to generate quick interactive molecular visualizations. To install the development version of bio3dview from GitHub, along with the related NGLVieweR package use:

```
install.packages("remotes")       remotes::install_github("bioboot/bio3dview")
install.packages("NGLVieweR")
```

Then load the respective packages and generate a quick NGL (webGL based) structure overview of a bio3d pdb class object with a number of simple defaults. The returned NGLVieweR object can be further added to build custom interactive visualizations:

```
library(bio3dview)
library(NGLVieweR)

view.pdb(pdb) |>
  setSpin()
```

You can also customize the display in many ways with minimal code. For example, lets custom color the chains and highlight some key residues as spacefill/vdw:

```
sele <- atom.select(pdb, resno=25)

view.pdb(pdb, cols=c("navy","teal"),
         highlight = sele,
         highlight.style = "spacefill") |>
  setRock()
```

Let's read a new PDB structure of Adenylate Kinase and perform Normal mode analysis.

```
adk <- read.pdb("6s36")
```

```
  Note: Accessing on-line PDB file
   PDB has ALT records, taking A only, rm.alt=TRUE
```

```
adk
```

```
 Call:  read.pdb(file = "6s36")

   Total Models#: 1
     Total Atoms#: 1898,  XYZs#: 5694  Chains#: 1  (values: A)
```

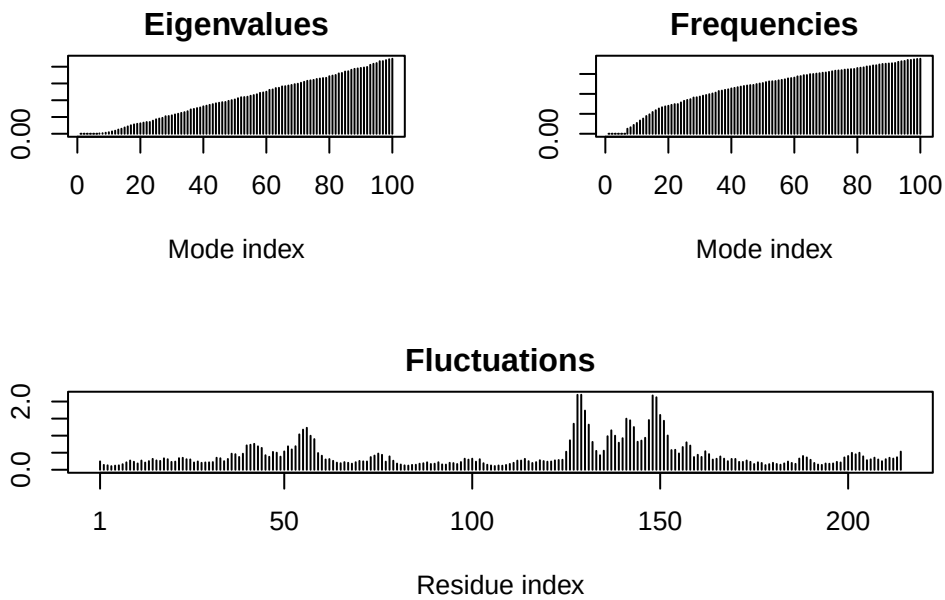
```
     Protein Atoms#: 1654  (residues/Calpha atoms#: 214)
     Nucleic acid Atoms#: 0  (residues/phosphate atoms#: 0)

     Non-protein/nucleic Atoms#: 244  (residues: 244)
     Non-protein/nucleic resid values: [ CL (3), HOH (238), MG (2), NA (1) ]

   Protein sequence:
      MRIILLGAPGAGKGTQAQFIMEKYGIPQISTGDMLRAAVKSGSELGKQAKDIMDAGKLVT
      DELVIALVKERIAQEDCRNGFLLDGFPRTIPQADAMKEAGINVDYVLEFDVPDELIVDKI
      VGRRVHAPSGRVYHVKFNPPKVEGKDDVTGEELTTRKDDQEETVRKRLVEYHQMTAPLIG
      YYSKEAEAGNTKYAKVDGTKPVAEVRADLEKILG

+ attr: atom, xyz, seqres, helix, sheet,
        calpha, remark, call
```

Normal mode analysis (NMA) is a structural bioinformatics method to predict protein flexibility and potential functional motions (a.k.a. conformational changes).

```r
m <- nma(adk)
```

```
 Building Hessian...        Done in 0.013 seconds.
 Diagonalizing Hessian...   Done in 0.079 seconds.
```

```r
plot(m)
```



To view a "movie" of these predicted motions we can generate a molecular "trajectory" with the mktrj() function.

```
mktrj(m, file="adk_m7.pdb")
```

Alternatively, for a quicker display you can use the view.nma() function from the bio3dview package mentioned previously:

```
view.nma(m, pdb=adk)
```

Comparative structure analysis of Adenylate Kinase

We will start with a database identifier (id) "1ake_A":

```
library(bio3d)

id <- "1ake_A"
aa <- get.seq(id)
```

```
Warning in get.seq(id): Removing existing file: seqs.fasta
```

```
Fetching... Please wait. Done.
```

```r
blast <- blast.pdb(aa)
```

```
 Searching ... please wait (updates every 5 seconds) RID = STN52XFW016
 ..............................................................
 Reporting 96 hits
```

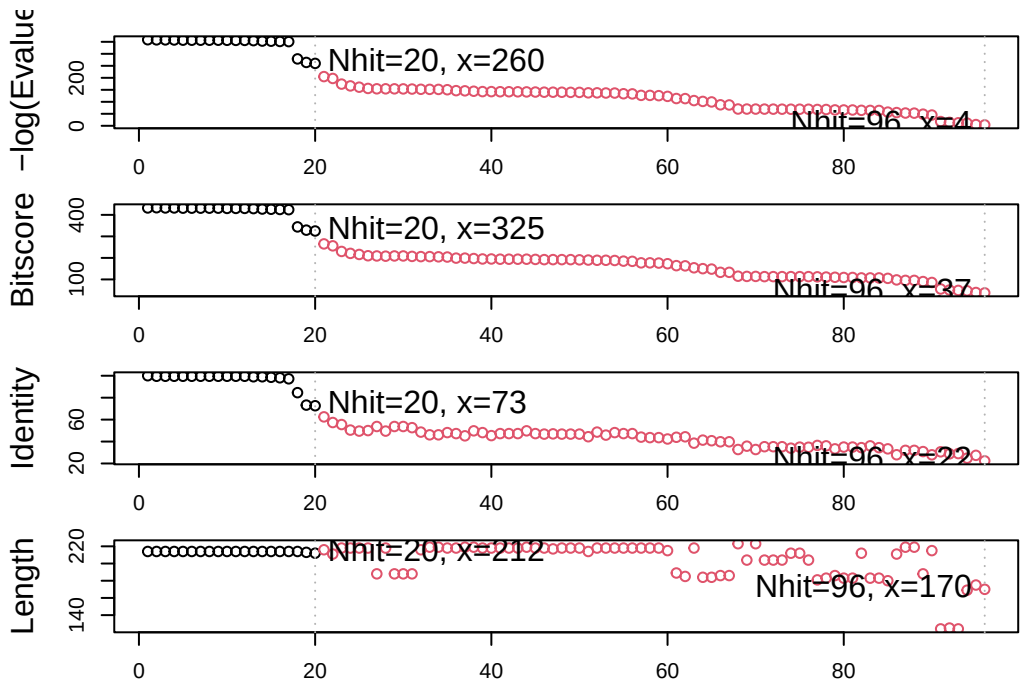
Have a peak:

```r
head(blast$hit.tbl)
```

```
        queryid subjectids identity alignmentlength mismatches gapopens q.start
1 Query_5191211     1AKE_A  100.000             214          0        0       1
2 Query_5191211     8BQF_A   99.533             214          1        0       1
3 Query_5191211     4X8M_A   99.533             214          1        0       1
4 Query_5191211     6S36_A   99.533             214          1        0       1
5 Query_5191211     9R6U_A   99.533             214          1        0       1
6 Query_5191211     9R71_A   99.533             214          1        0       1
  q.end s.start s.end    evalue bitscore positives mlog.evalue pdb.id    acc
1   214       1   214 1.78e-156      432    100.00    358.6267 1AKE_A 1AKE_A
2   214      21   234 2.91e-156      433    100.00    358.1351 8BQF_A 8BQF_A
3   214       1   214 3.18e-156      432    100.00    358.0464 4X8M_A 4X8M_A
4   214       1   214 4.67e-156      432    100.00    357.6621 6S36_A 6S36_A
5   214       1   214 1.04e-155      431     99.53    356.8615 9R6U_A 9R6U_A
6   214       1   214 1.23e-155      431     99.53    356.6937 9R71_A 9R71_A
```

```r
hits <- plot(blast)
```

```
  * Possible cutoff values:    260 3
            Yielding Nhits:    20 96

  * Chosen cutoff value of:    260
            Yielding Nhits:    20
```



Peak at out top hits:

```
head(hits$pdb.id)
```

```
[1] "1AKE_A" "8BQF_A" "4X8M_A" "6S36_A" "9R6U_A" "9R71_A"
```

Now we can download these "top hits", these will all be ADK (Adenylyl Kinase) structures in the PDB database.

```
files <- get.pdb(hits$pdb.id, path="pdbs", split=TRUE, gzip=TRUE)
```

```
Warning in get.pdb(hits$pdb.id, path = "pdbs", split = TRUE, gzip = TRUE):
pdbs/1AKE.pdb.gz exists. Skipping download

Warning in get.pdb(hits$pdb.id, path = "pdbs", split = TRUE, gzip = TRUE):
pdbs/8BQF.pdb.gz exists. Skipping download

Warning in get.pdb(hits$pdb.id, path = "pdbs", split = TRUE, gzip = TRUE):
pdbs/4X8M.pdb.gz exists. Skipping download

Warning in get.pdb(hits$pdb.id, path = "pdbs", split = TRUE, gzip = TRUE):
pdbs/6S36.pdb.gz exists. Skipping download
```

```
Warning in get.pdb(hits$pdb.id, path = "pdbs", split = TRUE, gzip = TRUE):
pdbs/9R6U.pdb.gz exists. Skipping download

Warning in get.pdb(hits$pdb.id, path = "pdbs", split = TRUE, gzip = TRUE):
pdbs/9R71.pdb.gz exists. Skipping download

Warning in get.pdb(hits$pdb.id, path = "pdbs", split = TRUE, gzip = TRUE):
pdbs/8Q2B.pdb.gz exists. Skipping download

Warning in get.pdb(hits$pdb.id, path = "pdbs", split = TRUE, gzip = TRUE):
pdbs/8RJ9.pdb.gz exists. Skipping download

Warning in get.pdb(hits$pdb.id, path = "pdbs", split = TRUE, gzip = TRUE):
pdbs/6RZE.pdb.gz exists. Skipping download

Warning in get.pdb(hits$pdb.id, path = "pdbs", split = TRUE, gzip = TRUE):
pdbs/4X8H.pdb.gz exists. Skipping download

Warning in get.pdb(hits$pdb.id, path = "pdbs", split = TRUE, gzip = TRUE):
pdbs/3HPR.pdb.gz exists. Skipping download

Warning in get.pdb(hits$pdb.id, path = "pdbs", split = TRUE, gzip = TRUE):
pdbs/1E4V.pdb.gz exists. Skipping download

Warning in get.pdb(hits$pdb.id, path = "pdbs", split = TRUE, gzip = TRUE):
pdbs/5EJE.pdb.gz exists. Skipping download

Warning in get.pdb(hits$pdb.id, path = "pdbs", split = TRUE, gzip = TRUE):
pdbs/1E4Y.pdb.gz exists. Skipping download

Warning in get.pdb(hits$pdb.id, path = "pdbs", split = TRUE, gzip = TRUE):
pdbs/3X2S.pdb.gz exists. Skipping download

Warning in get.pdb(hits$pdb.id, path = "pdbs", split = TRUE, gzip = TRUE):
pdbs/6HAP.pdb.gz exists. Skipping download

Warning in get.pdb(hits$pdb.id, path = "pdbs", split = TRUE, gzip = TRUE):
pdbs/6HAM.pdb.gz exists. Skipping download
```

```
Warning in get.pdb(hits$pdb.id, path = "pdbs", split = TRUE, gzip = TRUE):
pdbs/8PVW.pdb.gz exists. Skipping download

Warning in get.pdb(hits$pdb.id, path = "pdbs", split = TRUE, gzip = TRUE):
pdbs/4K46.pdb.gz exists. Skipping download

Warning in get.pdb(hits$pdb.id, path = "pdbs", split = TRUE, gzip = TRUE):
pdbs/4NP6.pdb.gz exists. Skipping download


  |
  |                                                                      |   0%
  |
  |====                                                                  |   5%
  |
  |=======                                                               |  10%
  |
  |==========                                                            |  15%
  |
  |==============                                                        |  20%
  |
  |==================                                                    |  25%
  |
  |======================                                                |  30%
  |
  |========================                                              |  35%
  |
  |============================                                          |  40%
  |
  |================================                                      |  45%
  |
  |====================================                                  |  50%
  |
  |========================================                              |  55%
  |
  |============================================                          |  60%
  |
  |================================================                      |  65%
  |
  |====================================================                  |  70%
  |
  |========================================================              |  75%
```

```
  |
  |=========================================================             |  80%
  |
  |==========================================================           |  85%
  |
  |=============================================================        |  90%
  |
  |==================================================================   |  95%
  |
  |======================================================================| 100%
```

We need one package from BioConductor. To set this up, we need to first install a package called **"BiocManager"** from CRAN.

Now we can use the `insall()` function from this package like this: `BiocManager::install("msa")`

```r
pdbs <- pdbaln(files, fit = TRUE, exefile= "msa")
```

```
Reading PDB files:
pdbs/split_chain/1AKE_A.pdb
pdbs/split_chain/8BQF_A.pdb
pdbs/split_chain/4X8M_A.pdb
pdbs/split_chain/6S36_A.pdb
pdbs/split_chain/9R6U_A.pdb
pdbs/split_chain/9R71_A.pdb
pdbs/split_chain/8Q2B_A.pdb
pdbs/split_chain/8RJ9_A.pdb
pdbs/split_chain/6RZE_A.pdb
pdbs/split_chain/4X8H_A.pdb
pdbs/split_chain/3HPR_A.pdb
pdbs/split_chain/1E4V_A.pdb
pdbs/split_chain/5EJE_A.pdb
pdbs/split_chain/1E4Y_A.pdb
pdbs/split_chain/3X2S_A.pdb
pdbs/split_chain/6HAP_A.pdb
pdbs/split_chain/6HAM_A.pdb
pdbs/split_chain/8PVW_A.pdb
pdbs/split_chain/4K46_A.pdb
pdbs/split_chain/4NP6_A.pdb
   PDB has ALT records, taking A only, rm.alt=TRUE
.   PDB has ALT records, taking A only, rm.alt=TRUE
..   PDB has ALT records, taking A only, rm.alt=TRUE
```

```
.   PDB has ALT records, taking A only, rm.alt=TRUE
.   PDB has ALT records, taking A only, rm.alt=TRUE
.   PDB has ALT records, taking A only, rm.alt=TRUE
.   PDB has ALT records, taking A only, rm.alt=TRUE
.   PDB has ALT records, taking A only, rm.alt=TRUE
..   PDB has ALT records, taking A only, rm.alt=TRUE
..   PDB has ALT records, taking A only, rm.alt=TRUE
....   PDB has ALT records, taking A only, rm.alt=TRUE
.   PDB has ALT records, taking A only, rm.alt=TRUE
.   PDB has ALT records, taking A only, rm.alt=TRUE
..

Extracting sequences

pdb/seq: 1   name: pdbs/split_chain/1AKE_A.pdb
   PDB has ALT records, taking A only, rm.alt=TRUE
pdb/seq: 2   name: pdbs/split_chain/8BQF_A.pdb
   PDB has ALT records, taking A only, rm.alt=TRUE
pdb/seq: 3   name: pdbs/split_chain/4X8M_A.pdb
pdb/seq: 4   name: pdbs/split_chain/6S36_A.pdb
   PDB has ALT records, taking A only, rm.alt=TRUE
pdb/seq: 5   name: pdbs/split_chain/9R6U_A.pdb
   PDB has ALT records, taking A only, rm.alt=TRUE
pdb/seq: 6   name: pdbs/split_chain/9R71_A.pdb
   PDB has ALT records, taking A only, rm.alt=TRUE
pdb/seq: 7   name: pdbs/split_chain/8Q2B_A.pdb
   PDB has ALT records, taking A only, rm.alt=TRUE
pdb/seq: 8   name: pdbs/split_chain/8RJ9_A.pdb
   PDB has ALT records, taking A only, rm.alt=TRUE
pdb/seq: 9   name: pdbs/split_chain/6RZE_A.pdb
   PDB has ALT records, taking A only, rm.alt=TRUE
pdb/seq: 10   name: pdbs/split_chain/4X8H_A.pdb
pdb/seq: 11   name: pdbs/split_chain/3HPR_A.pdb
   PDB has ALT records, taking A only, rm.alt=TRUE
pdb/seq: 12   name: pdbs/split_chain/1E4V_A.pdb
pdb/seq: 13   name: pdbs/split_chain/5EJE_A.pdb
   PDB has ALT records, taking A only, rm.alt=TRUE
pdb/seq: 14   name: pdbs/split_chain/1E4Y_A.pdb
pdb/seq: 15   name: pdbs/split_chain/3X2S_A.pdb
pdb/seq: 16   name: pdbs/split_chain/6HAP_A.pdb
pdb/seq: 17   name: pdbs/split_chain/6HAM_A.pdb
   PDB has ALT records, taking A only, rm.alt=TRUE
pdb/seq: 18   name: pdbs/split_chain/8PVW_A.pdb
```

```
   PDB has ALT records, taking A only, rm.alt=TRUE
pdb/seq: 19   name: pdbs/split_chain/4K46_A.pdb
   PDB has ALT records, taking A only, rm.alt=TRUE
pdb/seq: 20   name: pdbs/split_chain/4NP6_A.pdb
```

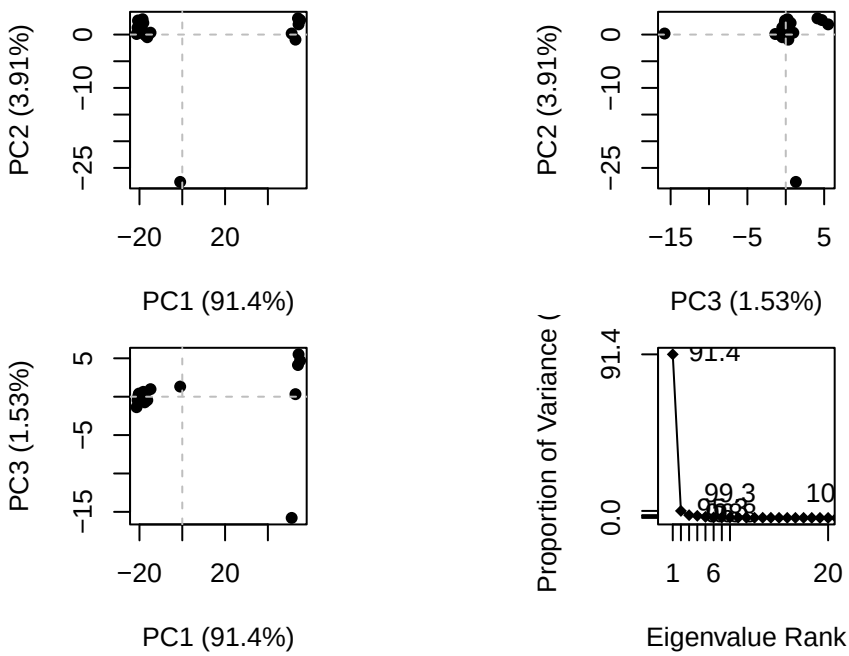
Let's have a wee peak at our structures after "fitting" or superposing

```
#library(bio3dview)
#view.pdbs(pdbs)
```

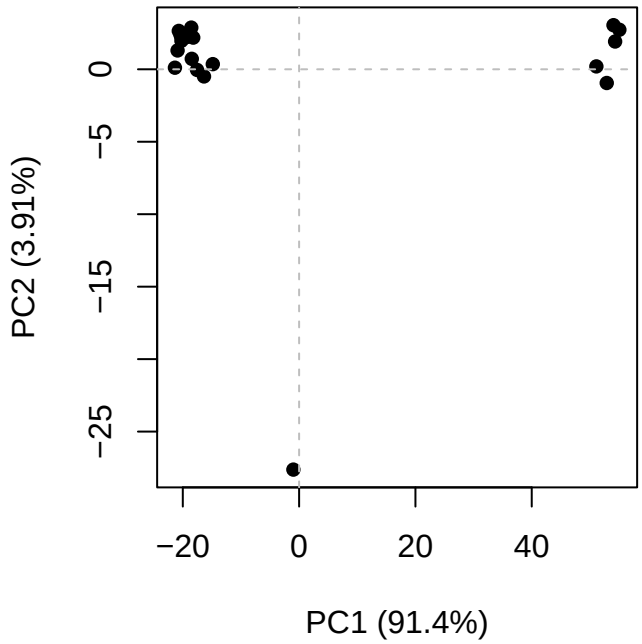
```
#view.pdbs(pdbs, colorScheme = "residue")
```

We can run functions like `rmsd()`, `rmsf()` and `pca()`

```
pc.xray <- pca(pdbs)
plot(pc.xray)
```




```
plot(pc.xray, 1:2)
```



Finally, let's make a wee movie of the major "motion" or structural difference in the dataset- we call this a trajectory"

```
mktrj(pc.xray, file= "results.pdb")
```

Q10: Which package is only on Bioconductor?

msa

Q11: Which package is NOT on CRAN or Bioconductor?

bio3dview

Q12: True or False — pak can install from GitHub & BitBucket?

True

Q13. How many amino acids are in this sequence, i.e. how long is this sequence?

```
library(bio3d)
aa <- get.seq("1ake_A")
```

```
Warning in get.seq("1ake_A"): Removing existing file: seqs.fasta
```

```
Fetching... Please wait. Done.
```

```
aa
```

```
             1        .         .         .         .         .         60
pdb|1AKE|A   MRIILLGAPGAGKGTQAQFIMEKYGIPQISTGDMLRAAVKSGSELGKQAKDIMDAGKLVT
             1        .         .         .         .         .         60

            61        .         .         .         .         .         120
pdb|1AKE|A   DELVIALVKERIAQEDCRNGFLLDGFPRTIPQADAMKEAGINVDYVLEFDVPDELIVDRI
            61        .         .         .         .         .         120

           121        .         .         .         .         .         180
pdb|1AKE|A   VGRRVHAPSGRVYHVKFNPPKVEGKDDVTGEELTTRKDDQEETVRKRLVEYHQMTAPLIG
           121        .         .         .         .         .         180

           181        .         .         .   214
pdb|1AKE|A   YYSKEAEAGNTKYAKVDGTKPVAEVRADLEKILG
           181        .         .         .   214

Call:
  read.fasta(file = outfile)

Class:
  fasta

Alignment dimensions:
  1 sequence rows; 214 position columns (214 non-gap, 0 gap)

+ attr: id, ali, call
```

214 position columns

> Q14. What do you note about this plot? Are the black and colored lines similar or different? Where do you think they differ most and why?

The black and colored lines are similar, indicating that the adenylate kinase structures share a common core fold. The largest differences occur at specific regions with higher fluctuations, specifically around loop regions and hinge areas. These regions are known to be more flexible and are involved in conformational changes required for enzyme activity, which explains the increased variability across structures.